

# Visualisation avec

Dernière mise à jour : 02 oct. 2026

Sandrine LYSER

## Table des matières

|                                                                                                           |           |
|-----------------------------------------------------------------------------------------------------------|-----------|
| Présentation du cours                                                                                     | 1         |
| Objectifs pédagogiques                                                                                    | 1         |
| Organisation des séances                                                                                  | 1         |
| Évaluation finale                                                                                         | 2         |
| <b>1 Séance 1. R et RStudio, bonnes pratiques pour une analyse reproductible : du projet au tidyverse</b> | <b>2</b>  |
| 1.1 Objectifs pédagogiques                                                                                | 2         |
| 1.2 La reproductibilité                                                                                   | 2         |
| 1.2.1 Pourquoi parler de reproductibilité ?                                                               | 2         |
| 1.2.2 Qu'est-ce qu'une analyse reproductible ?                                                            | 3         |
| 1.2.3 Que se passe-t'il sans "bonnes" pratiques ?                                                         | 3         |
| 1.2.4 De la donnée au résultat : une chaîne de travail transparente et documentée                         | 4         |
| 1.3 Comment travailler de façon reproductible avec R/RStudio ?                                            | 4         |
| 1.3.1 Les projets RStudio                                                                                 | 4         |
| 1.3.2 Les scripts R                                                                                       | 6         |
| 1.3.3 Le fichier README                                                                                   | 8         |
| 1.3.4 Citer la version de R ainsi que les packages que vous avez utilisés dans vos travaux                | 9         |
| 1.4 À VOUS DE JOUER !                                                                                     | 10        |
| 1.5 L'univers du tidyverse                                                                                | 10        |
| 1.5.1 Présentation générale et installation                                                               | 10        |
| 1.6 Manipulations des données                                                                             | 12        |
| 1.6.1 Import/Export des données                                                                           | 12        |
| 1.6.2 Manipulations sur les observations et les variables avec le package [dplyr]                         | 13        |
| 1.7 À VOUS DE JOUER !                                                                                     | 18        |
| <b>2 Bibliographie</b>                                                                                    | <b>19</b> |

## Présentation du cours

### Objectifs pédagogiques

Dans ce cours, vous apprendrez à...

- utiliser R sous RStudio en utilisant les projets RStudio
- travailler de manière **reproductible**, en organisant votre travail et en structurant vos scripts
- utiliser les principaux outils du **tidyverse** pour manipuler et visualiser des données
- produire des visualisations de données sous forme de graphiques de qualité, en utilisant [ggplot2](#)

### Organisation des séances

1. Vendredi 02 octobre 2026 (3h)  
R et RStudio, bonnes pratiques pour une analyse reproductible : du projet au tidyverse

2. Lundi 05 octobre 2026 (2h30)  
Premiers graphiques [\[ggplot2\]](#)  
Transformation des données et approfondissements [\[ggplot2\]](#)
3. Vendredi 09 octobre 2026 (4h30)  
Visualisations avancées
4. Vendredi 02 novembre (2h)  
Évaluation finale (1h30)  
Débriefing évaluation et cours

## Évaluation finale

Format pratique sur ordinateur pour évaluer

- l'organisation du code
- la pertinence graphique
- la bonne utilisation du `tidyverse`
- la clarté/reproductibilité du script

## 📊 Données utilisées dans les exemples

Les exemples du cours s'appuient sur le jeu de données `Journals`, disponible dans le package `{Ecdat}`

Ce jeu de données se compose de 180 observations décrites par 10 variables

- **title** : titre de la revue
- **pub** : éditeur
- **society** : société
- **libprice** : prix de l'abonnement
- **pages** : nombre de pages
- **charpp** : nombre de caractères par page
- **citestot** : nombre total de citations
- **date1** : année de création de la revue
- **oclc** : nombre d'abonnements
- **field** : domaine de la revue

## 📊 Données utilisées dans les exercices

Les exercices se feront sur le jeu de données fictives `communes`, disponible dans l'espace `elearn`

# 1 Séance 1. R et RStudio, bonnes pratiques pour une analyse reproductible : du projet au tidyverse

## 1.1 Objectifs pédagogiques

À l'issue de cette séance, vous serez capable de

- comprendre ce qu'est la reproductibilité
- de mettre en oeuvre de "bonnes" pratiques sous R/RStudio pour répondre aux enjeux de reproductibilité
- créer et organiser un projet RStudio, en structurant vos scripts
- importer un jeu de données
- effectuer les premières manipulations et transformations sur les données avec le package `[dplyr]`

## 1.2 La reproductibilité

### 1.2.1 Pourquoi parler de reproductibilité ?

Une analyse ne se résume pas à son résultat, celui-ci doit pouvoir être

- compris
- vérifié

- réexécuté
- corrigé
- mis à jour
- partagé avec d'autres personnes

### 1.2.2 Qu'est-ce qu'une analyse reproductible ?

Une analyse est dite **reproductible** lorsque l'on peut obtenir les mêmes résultats en utilisant

- les mêmes données en entrée
- les mêmes méthodes
- les mêmes étapes de calcul
- le même code

**La reproductibilité est un processus qui peut être reproduit par une autre personne et/ou à partir des mêmes données et qui permet d'expliquer et partager son travail**

#### **i** Reproductibilité ? Réplicabilité ?

Deux notions à distinguer

- **reproductibilité** : en utilisant le même code et les mêmes données , on retrouve le même résultat (valeur numérique, tableau, graphique, etc.)
- **réplicabilité** : en appliquant la même démarche, sur de nouvelles données, on obtient des résultats comparables

### 1.2.3 Que se passe-t'il sans "bonnes" pratiques ?

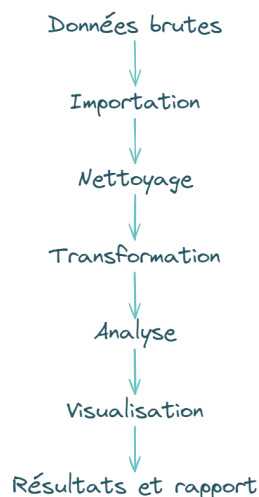
- Des résultats impossibles à retrouver
- Des chemins de fichiers propres à l'ordinateur de l'auteur
- Des modifications manuelles sur le fichier de données non documentées
- Des commandes exécutées dans la console qui ne figurent pas dans le script
- Des erreurs de copier-coller ou des résultats copiés manuellement dans un rapport
- Des versions différentes de R ou des packages
- Des difficultés pour identifier les packages utilisés
- Des difficultés à transmettre ou à reprendre un travail plusieurs mois plus tard

#### **!** Ces problèmes sont souvent liés à l'organisation du travail

#### **>** Trois grands principes à respecter pour éviter ces erreurs (Orozco et al., 2020)

- Organiser son travail
- Coder pour les autres
- Automatiser le plus possible

### 1.2.4 De la donnée au résultat : une chaîne de travail transparente et documentée



À chaque étape, il faut pouvoir répondre à trois questions

1. Qu'a-t-on fait ?
2. Pourquoi l'a-t-on fait ?
3. Peut-on refaire cette étape ?

» Chaque opération doit être identifiable, justifiée et réexécutable

La reproductibilité repose sur la conservation

- des données
- du flux de traitement (scripts d'importation et de traitement, fonctions, méthodes statistiques utilisées)
- de la documentation (métadonnées, dictionnaire des variables, version de R et des packages, instructions pour relancer l'analyse)
- des informations nécessaires à l'interprétation (tableaux, graphiques)

## 1.3 Comment travailler de façon reproductible avec R/RStudio ?

### 1.3.1 Les projets RStudio

Les projets RStudio sont utiles pour

- **organiser plus facilement son travail** : 1 projet = 1 étude
  - » environnement isolé, pas de conflits entre projets
- **regrouper dans un même dossier tous les scripts** : scripts R/quarto, fonctions, données, résultats, graphiques, etc.
- **améliorer la portabilité et faciliter les collaborations** : utilisation des chemins de fichiers relatifs
  - » reproductibilité facilitée

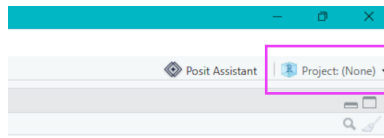
#### i Chemins absolus/relatifs

- Le chemin absolu est exprimé par rapport au disque dur, à l'ordinateur  
*Exemple* : C:/Users/...
- Le chemin relatif est exprimé par rapport au projet  
*Exemple* : data/mon\_fichier.csv

💡 Le répertoire par défaut du projet est le dossier où est enregistré le projet sur votre machine

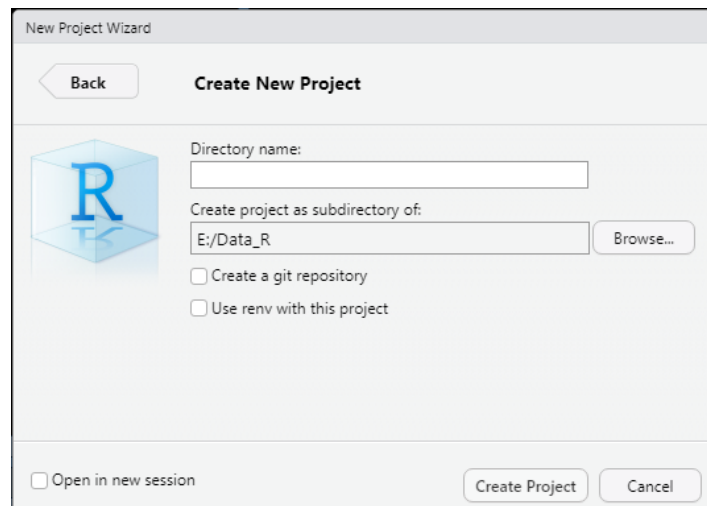
#### 1.3.1.1 Comment créer un projet ?

À partir de l'icône dédiée en haut à droite de RStudio



Cliquez sur New Project > New Directory > New Project et complétez les informations requises

- le nom du dossier/projet
- son emplacement (dossier de votre choix)



#### 💡 Quand le projet est créé

- RStudio redémarre et se place dans ce projet
- le projet apparaît dans l'onglet *Files* de RStudio

#### 💡 Pour ranger efficacement les différents documents liés à l'analyse, vous pouvez créer les sous-dossiers

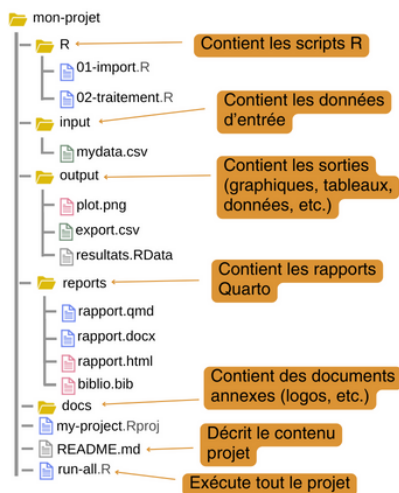
- soit en cliquant sur **New Folder** dans le menu en haut de l'onglet *Files*
- soit de façon classique, à partir de l'explorateur de fichier de votre machine

### 1.3.1.2 Comment structurer ses projets ?

Choisissez la structuration qui vous correspond le mieux et qui permet

- de **rassembler** tous les éléments liés au projet dans un seul dossier
- de **structurer** en dossiers/sous-dossiers pour séparer données source/créées, scripts, documentation, résultats, etc.

#### 💡 Les données initiales et les scripts de traitement sont les plus importants



```
name_of_project
|--data
|  |--2017report.csv
|  |--2016report.pdf
|  |--summary2016_2017.csv
|--docs
|  |--01-analysis.Rmd
|  |--01-analysis.html
|--scripts
|  |--exploratory_analysis.R
|  |--pdf_scraper.R
|--name_of_project.Rproj
|--run_all.R
```

```
name_of_project
|--raw_data
|  |--WhateverData.xlsx
|  |--2017report.csv
|  |--2016report.pdf
|--output_data
|  |--summary2016_2017.csv
|--rmd
|  |--01-analysis.Rmd
|--docs
|  |--01-analysis.html
|  |--01-analysis.pdf
|  |--02-deeper.html
|  |--02-deeper.pdf
|--scripts
|  |--exploratory_analysis.R
|  |--pdf_scraper.R
|--name_of_project.Rproj
|--run_all.R
```

Exemples de structuration de projets R Source (figures 2 & 3) : <https://learn.r-journalism.com/en/publishing/workflow/r-projects/> - Dernier accès : 2026-09-08

### 1.3.2 Les scripts R

- Ils permettent de **garder la trace** du processus de création de chaque variable, tableau, figure, etc., ce qui permet de refaire les traitements dans leur globalité
- Ils sont **facilement partageables** avec d'autres, sous réserve de respecter des pratiques de "bon" usage
  - coder de façon compréhensible avec des commentaires et en documentant ses fonctions
  - donner des noms explicites aux objets et fonctions
  - utiliser les conventions de nommage, notamment pour distinguer les variables d'origine des variables créées



- Pour les **variables**, utiliser un **nom**  
*Exemple* : temperature\_max
- Pour les **fonctions**, utiliser un **verbe**  
*Exemple* : create.map serait une fonction qui permettrait de créer une carte
- Pour **distinguer variables d'origine / variables recodées**, on peut utiliser un préfixe ou un suffixe  
*Exemple* : age est la variable d'origine et age\_recod est la variable recodée

#### i Les conventions de nommage

Il existe différentes conventions

- **alllowercase** : tout en minuscule, sans séparateur
- **period.separated** : tout en minuscule, mots séparés par des points
- **underscore\_separated** : tout en minuscule, mots séparés par un *underscore* (`_`)
- **lowerCamelCase** : première lettre des mots en majuscule, à l'exception du premier mot; et si nom simple, tout en minuscule
- **UpperCamelCase** : première lettre des mots en majuscule, y compris le premier et même lorsque le nom est composé d'un seul mot

💡 Possibilité de mixer les conventions mais en gardant une cohérence afin de faciliter la compréhension des fichiers et du code

⚠️ R est sensible à la casse ce qui signifie que variable et Variable sont deux objets différents

### 1.3.2.1 Comment organiser ses scripts ?

- Utiliser une **convention de dénomination des fichiers** et donner des **noms explicites**
- **Séparer les traitements**
  - import et préparation des données (nettoyage, fusion, filtrage)
  - traitement statistique
    - analyses univariées
    - analyses bivariées
    - analyses multivariées/modélisation
  - production des résultats, rapports

#### 💡 Comment nommer ses scripts ?

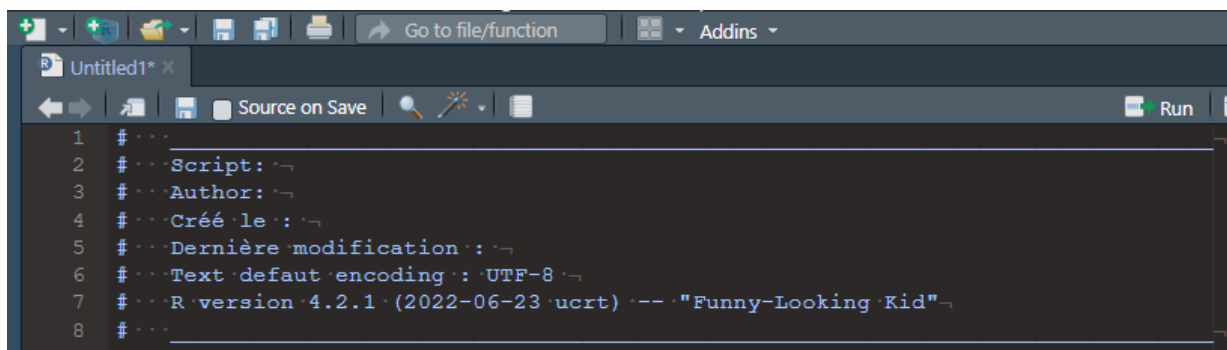
- Une "bonne" pratique consiste à utiliser la structure `numero_nom_millesime.extension`
- Le nom ne doit contenir ni caractères spéciaux ('-..,;:\/\$^') ni caractères accentués  
*Exemple* : `01_import_donnees_20250606.R`; `02_nettoyage_donnees_20250610.R`;  
`03_analyses_univariees_20250701.R`

#### i Ressources sur le processus d'analyse et de communication des données reproductibles

-  [R Workflow](#)
-  [Reproducible analytical pipeline](#)

### 1.3.2.2 Comment structurer ses scripts ?

1. Commencer l'écriture du script par les **métadonnées** (titre, auteur, date, version de R, encodage, etc.)



```
1 # ...
2 # ---Script: ---
3 # ---Author: ---
4 # ---Créé le : ---
5 # ---Dernière modification : ---
6 # ---Text default encoding : UTF-8 ---
7 # ---R version 4.2.1 (2022-06-23 ucrt) --- "Funny-Looking Kid"---
8 # ...
```

2. Lister les **packages** utilisés, au début des scripts, pour les identifier plus rapidement, en notant leur version

- manuellement

```
library(tidyverse)    # v2.0.0
library(questionr)    # v0.8.2
```

- de façon automatisée

```
packages_utilises <- sessionInfo()
```

3. **Structurer** en parties et sous-parties, facilement grâce à l'*addin strcode*

```
# -----
# Titre 1 #####
```

```
## .....
## Titre 2 #####
```

🔧 *Installation : `remotes::install_github("LorenzWalther/strcode")`*

4. **Commenter** les scripts, en particulier pour expliquer les étapes-clés (= une aide pour les autres, mais aussi pour soi, pour plus tard)

### i Le préfixage

La notation

```
dplyr::filter(Journals, field == "Econometrics")
```

peut remplacer le code

```
library(dplyr)
filter(Journals, field == "Econometrics")
```

#### Avantages

- Évite les conflits si plusieurs packages ont une fonction avec le même nom (mais des paramètres différents)  
*Exemple* : la fonction `filter()` existe dans les packages `[stats]` et `[dplyr]`
- Identifie le package d'où provient une fonction
- Ne charge pas la totalité d'un package pour n'utiliser qu'une seule de ses fonctions

### 💡 Aide pour écrire les scripts

🔧 Activer les options de diagnostic pour signaler les erreurs dans les scripts sous RStudio : **Tools > Global Options > Code > Diagnostics**

```
13 -
14 # -
15 # Packages ..... #####
16 library(tidyverse) ... # v1.3.2
17 -
```

🔧 package `[styler]` : mettre en forme le code selon le [Tidyverse Style Guide](#) (ou selon son propre style)

🔧 package `[lintr]` : vérifier que le code est conforme à un guide de style donné

### 1.3.3 Le fichier README

Ce fichier, généralement un fichier au format `.txt` ou `.md` (markdown), qui présente le **projet**, sa **structuration** et les **étapes d'exécution des différents script** pour comprendre, reproduire et utiliser le projet, est à placer dans le dossier du projet

#### Exemple de fichier README

```
# VisuR_2026
```

Projet du cours "Visualisation des données sous R" pour l'année universitaire 2026-2027

```
## Structure
```

```
- data/ : données
```

```

├─ sources/ : données sources, données brutes
├─ produites/ : données produites
- scripts/ : scripts R, qmd
- outputs/ : graphiques et tableaux exportés
- docs/ : rapports et supports

## Comment reproduire les analyses
1. Ouvrir le projet `VisuR_2026.Rproj`
2. Exécuter les scripts dans l'ordre (01_import.R, 02_nettoyage.R, etc.)
3. Les graphiques générés sont enregistrés dans le dossier outputs
4. Le rapport d'étude est enregistré dans le dossier docs

```

---

### 1.3.4 Citer la version de R ainsi que les packages que vous avez utilisés dans vos travaux

Avec la fonction `citation()`

#### 1.3.4.1 Pour citer la version de R

```

# Citer R
citation()

```

To cite R in publications use:

R Core Team (2026). *\_R: A Language and Environment for Statistical Computing\_*. R Foundation for Statistical Computing, Vienna, Austria.  
doi:10.32614/R.manuals <<https://doi.org/10.32614/R.manuals>>.  
<<https://www.R-project.org/>>.

Une entrée BibTeX pour les utilisateurs LaTeX est

```

@Manual{,
  title = {R: A Language and Environment for Statistical Computing},
  author = {{R Core Team}},
  organization = {R Foundation for Statistical Computing},
  address = {Vienna, Austria},
  year = {2026},
  doi = {10.32614/R.manuals},
  url = {https://www.R-project.org/},
}

```

We have invested a lot of time and effort in creating R, please cite it when using it for data analysis. See also '`citation("pkgname")`' for citing R packages.

#### 1.3.4.2 Pour citer un package

```

# Citer un package (ici ggplot2)
citation("ggplot2")

```

To cite ggplot2 in publications, please use

H. Wickham. *ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag New York, 2016.

Une entrée BibTeX pour les utilisateurs LaTeX est

```
@Book{,
  author = {Hadley Wickham},
  title = {ggplot2: Elegant Graphics for Data Analysis},
  publisher = {Springer-Verlag New York},
  year = {2016},
  isbn = {978-3-319-24277-4},
  url = {https://ggplot2.tidyverse.org},
}
```

## 1.4 🎮 À VOUS DE JOUER !

### Exercice 1.1.

1. Vérifier que [R] et [RStudio] sont installés
2. Définir l'encodage UTF-8
3. Vérifier que le package `{tidyverse}` est installé et l'installer si ce n'est pas le cas

**Quelle est la version de [R] et des packages `{tidyverse}` et `{ggplot2}` que vous utilisez ?**

4. Sous [RStudio], créer un projet nommé **VisuR\_exercices**
5. Créer les dossiers nécessaires pour une bonne structuration de votre travail
6. Déposer le fichier `communes.csv` dans le dossier adéquat de votre projet
7. Dans votre projet [RStudio], créer un script [R] nommé **01\_Exercices\_VisuR.R**, qui servira pour les exercices à venir, durant lesquels vous aurez besoin des packages `{readr}` et `{dplyr}`
8. Commencer l'écriture de ce script avec ces premiers éléments

## 1.5 L'univers du tidyverse

### 1.5.1 Présentation générale et installation

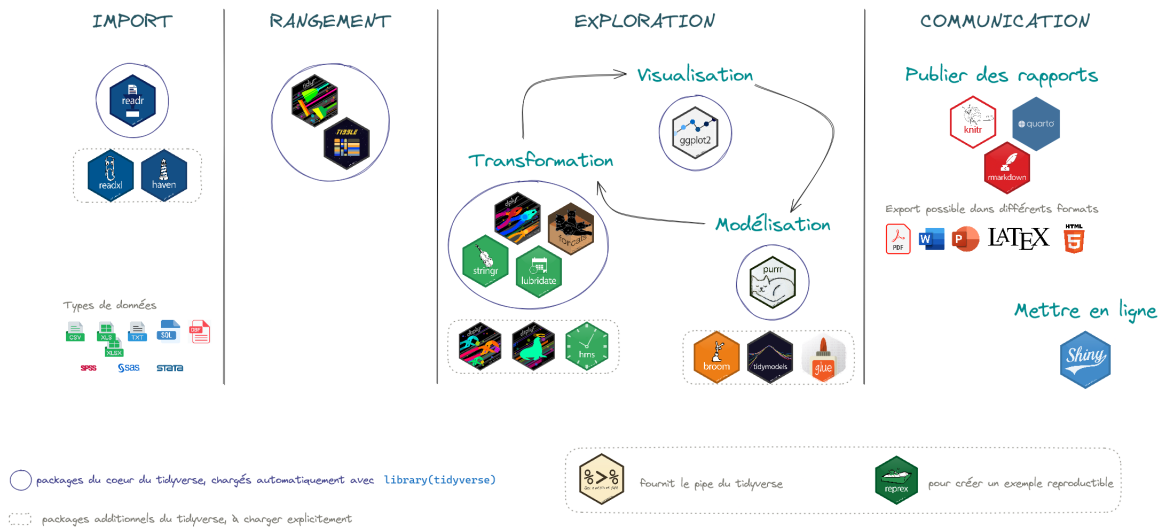
- **Ensemble de packages reprenant des opérations courantes de R, qui partagent une même philosophie et syntaxe pour travailler ensemble et faciliter, la manipulation, l'analyse et la visualisation des données**
- L'accent est mis sur la lisibilité et la chaîne d'opérations, via l'opérateur *pipe* `|>`  
 >> code dans l'ordre logique des étapes, moins d'objets intermédiaires et de parenthèses imbriquées que dans la syntaxe R de base

⚙️ La commande `install.packages("tidyverse")` installe simultanément plusieurs packages

- `{dplyr}` : manipulation des données
- `{forcats}` : traitement des variables qualitatives
- `{ggplot2}` : visualisation des données
- `{lubridate}` : manipulation des dates et heures
- `{purrr}` : programmation
- `{readr}` : import de données
- `{stringr}` : manipulation des chaînes de caractères
- `{tibble}` : version modernisée des *dataframes*, plus pratiques à utiliser que les *dataframes* "classiques"
- `{tidyr}` : nettoyage, remise en forme des données

💡 La commande `library("tidyverse")` charge simultanément ces neuf packages au coeur du tidyverse

## LES DIFFÉRENTES ÉTAPES DE LA DATA SCIENCE Les packages du Tidyverse & leurs principaux objectifs



### 1.5.1.1 L'opérateur pipe

- Opérateur qui permet d'enchaîner des opérations : il prend la sortie d'une fonction (ou d'une expression) et la passe automatiquement comme **premier** argument à la fonction suivante
- Initialement fourni dans le package `magrittr` et noté `%>%`
- Implémenté dans le langage **R** de base depuis la version 4.1.0 et noté `|>`

```
data |>
  fonction1() |>
  fonction2() |>
  fonction3()
# équivaut à
fonction3(fonction2(fonction1(data)))
```

💡 Il est recommandé d'utiliser le pipe natif de R `|>`

⚙️ Comment utiliser le pipe natif sous RStudio ?

- Allez dans Tools > Global Options > Code > Editing
- Cochez Use native pipe operator, `|>` (requires R 4.1+)

### 1.5.1.2 Les tibbles

- Version modernisée des **dataframes**, proposée dans le cadre du **tidyverse**
- Plus pratiques à utiliser que les **dataframes** "classiques"
- Plus rapides à manipuler (et plus stables) que les **dataframes**
- Il est possible de convertir d'un type à l'autre
  - **dataframe** → **tibble** : avec la fonction `as_tibble()` du package `tibble`
  - **tibble** → **dataframe** : avec la fonction **R** de base `as.data.frame()`

🔍 Pour connaître le type d'objet et savoir si on a un **dataframe** ou un **tibble**, on peut utiliser la fonction `str()`

## Aperçu des formats

### dataframe

```
Journals |>
  head(2)
```

```

      title                                pub society
1   Asian-Pacific Economic Literature      Blackwell      no
2 South African Journal of Economic History So Afr ec history assn      no
  libprice pages charpp citestot date1 oclc   field
1     123   440   3822         21 1986   14  General
2      20    309   1782         22 1986   59 Ec History
```

**tibble**

```
Journals |>
  as_tibble() |>
  head(2)
```

# A tibble: 2 x 10

```

  title          pub  society libprice pages charpp citestot date1 oclc field
  <fct>          <fct> <fct>    <int> <int>  <int>    <int> <int> <int> <fct>
1 Asian-Pacific ~ Blac~ no         123   440   3822         21 1986   14 Gene~
2 South African ~ So A~ no          20    309   1782         22 1986   59 Ec H~
```

## 1.6 Manipulations des données

### 1.6.1 Import/Export des données

- Depuis/vers des formats "plats" ou délimités (.csv, .txt)
- Depuis/vers des fichiers Excel ou issus d'autres logiciels de statistique (Stata, SPSS, etc.)
- Depuis/vers des bases de données relationnelles

💡 Pour l'import, bien étudier le jeu de données source, à l'aide des questions suivantes

1. La première ligne contient-elle le nom des variables ?
2. Quel est le séparateur des colonnes : , , ; , espace(s), \t ?
3. Quel est le caractère utilisé pour indiquer les décimales : le point (à l'anglo-saxonne) ou la virgule (à la française) ?
4. Les valeurs textuelles sont-elles encadrées par des guillemets ? Si oui, s'agit-il de guillemets simples (') ou de guillemets doubles (") ?
5. Y a-t-il des valeurs manquantes ? Si oui, comment sont-elles indiquées ?

#### 1.6.1.1 Les fonctions d'import

| Type de fichier | Séparateur de colonnes | Séparateur décimal | Tidyverse                         | Base R                            |
|-----------------|------------------------|--------------------|-----------------------------------|-----------------------------------|
| Délimité        | ,                      | .                  | <code>readr::read_csv()</code>    | <code>read.csv()</code>           |
|                 | ;                      | ,                  | <code>readr::read_csv2()</code>   | <code>read.csv2()</code>          |
|                 | un espace (ou plus)    | .                  | <code>readr::read_table()</code>  | <code>read.table()</code>         |
|                 | \t                     | .                  | <code>readr::read_delim()</code>  | <code>read.delim()</code>         |
|                 | \t                     | ,                  | <code>readr::read_delim2()</code> | <code>read.delim2()</code>        |
| Excel           |                        |                    | <code>readxl::read_excel()</code> | <code>xlsx::read.xlsx()</code>    |
| SPSS            |                        |                    | <code>haven::read_spss()</code>   | <code>foreign::read.spss()</code> |

| Type de fichier | Séparateur de colonnes | Séparateur décimal | Tidyverse                        | Base R                           |
|-----------------|------------------------|--------------------|----------------------------------|----------------------------------|
| Stata           |                        |                    | <code>haven::read_sav()</code>   |                                  |
| SAS             |                        |                    | <code>haven::read_stata()</code> | <code>foreign::read.dta()</code> |
| dBase           |                        |                    | <code>haven::read_sas()</code>   | <code>foreign::read.ssd()</code> |
|                 |                        |                    |                                  | <code>foreign::read.dbf()</code> |

### 1.6.1.2 Les fonctions d'export

| Type de fichier | Tidyverse                                                                  | Base R                                                |
|-----------------|----------------------------------------------------------------------------|-------------------------------------------------------|
| .txt            | <code>readr::write_delim(delim = "\t")</code>                              | <code>write.table()</code>                            |
| .csv            | <code>readr::write_csv()</code>                                            | <code>write.csv()</code>                              |
| .xlsx           | <code>openxlsx2::write_xlsx()</code><br><code>writexl::write_xlsx()</code> |                                                       |
| .dbf            |                                                                            | <code>foreign::write.dbf()</code>                     |
| .sav (SPSS)     | <code>haven::write_sav()</code>                                            | <code>foreign::write.foreign(package = "SPSS")</code> |
| .dta (Stata)    | <code>haven::write_dta()</code>                                            | <code>foreign::write.dta()</code>                     |

### 1.6.1.3 Exemple d'import pour un fichier .csv

```
readr::read_csv("file.csv"
  # si la première ligne contient le nom des variables :
  , col_names = TRUE)
```

#### Zoom sur la fonction `read_csv2()`

La fonction `readr::read_csv2()` considère par défaut que le séparateur de colonnes est le point-virgule et que la virgule sert de séparateur décimal

La fonction en langage R de base `read.csv2()` a les mêmes paramètres par défaut

 **Utiliser ces fonctions lorsque les fichiers .csv ont ces caractéristiques**

## 1.6.2 Manipulations sur les observations et les variables avec le package `ldplyr`

### Principales fonctions utiles pour

- **les lignes/observations**
  - `filter()` : sélectionner/filtrer des observations
  - `distinct` : afficher les observations distinctes
- **les colonnes/variables**
  - `select()` : sélectionner/extraire des variables
  - `mutate()` : créer des variables
  - `relocate()` : changer l'ordre d'une colonne
  - `rename()` : changer le nom des variables

 Aide-mémoire des principales fonctions du package `ldplyr`

 [Cheatsheet](#)

### 1.6.2.1 Filtrer des observations avec la fonction `filter()`

### 1.6.2.1.1 Selon un seul critère

```
# Revues dans Le domaine de La Démographie
Journals |>
  dplyr::filter(field == "Demography")
```

|     |        | title                           | pub                | society | libprice   | pages |
|-----|--------|---------------------------------|--------------------|---------|------------|-------|
| 30  |        | Journal of Population Economics | Springer           | no      | 411        | 640   |
| 116 |        | Population & Development Review | Population Council | no      | 36         | 910   |
| 137 |        | Demography                      | Pop Assn America   | no      | 85         | 568   |
|     | charpp | citestot                        | date1              | oclc    | field      |       |
| 30  | 3264   | 69                              | 1987               | 27      | Demography |       |
| 116 | 2904   | 445                             | 1975               | 218     | Demography |       |
| 137 | 6859   | 670                             | 1964               | 413     | Demography |       |

### 1.6.2.1.2 Selon plusieurs critères

En utilisant les opérateurs logiques

```
# Revues de Démographie dont l'abonnement coûte moins de 100$
Journals |>
  dplyr::filter(field == "Demography" & libprice < 100)
```

|     |        | title                           | pub                | society | libprice   | pages |
|-----|--------|---------------------------------|--------------------|---------|------------|-------|
| 116 |        | Population & Development Review | Population Council | no      | 36         | 910   |
| 137 |        | Demography                      | Pop Assn America   | no      | 85         | 568   |
|     | charpp | citestot                        | date1              | oclc    | field      |       |
| 116 | 2904   | 445                             | 1975               | 218     | Demography |       |
| 137 | 6859   | 670                             | 1964               | 413     | Demography |       |

#### i Opérateurs logiques les plus courants

| Opérateur | Description                    |
|-----------|--------------------------------|
| <         | Inférieur à                    |
| <=        | Inférieur ou égal à            |
| >         | Supérieur à                    |
| >=        | Supérieur ou égal à            |
| ==        | Exactement égal à              |
| !=        | Différent de                   |
|           | Ou                             |
| &         | Et                             |
| %in%      | Appartient à                   |
| is.na()   | Est une donnée manquante       |
| !is.na()  | N'est pas une donnée manquante |

### 1.6.2.2 Afficher les valeurs distinctes avec la fonction distinct()

On peut afficher les valeurs distinctes selon une (ou plusieurs) variables

```
# Afficher les valeurs distinctes de 'field'
Journals |>
  distinct(field)
```

|     | field              |
|-----|--------------------|
| 1   | General            |
| 2   | Ec History         |
| 3   | Specialized        |
| 4   | Area Studies       |
| 5   | Interdisciplinary  |
| 6   | Labor              |
| 7   | Development        |
| 11  | Consumer Ec        |
| 14  | Management Science |
| 15  | Finance            |
| 25  | Insurance          |
| 28  | Public Finance     |
| 29  | Urban and Regional |
| 30  | Demography         |
| 32  | Ind Organization   |
| 36  | Business           |
| 39  | Econometrics       |
| 40  | Ag Econ            |
| 45  | Natural Resources  |
| 57  | Law and Econ       |
| 58  | International      |
| 59  | Macroeconomics     |
| 84  | Theory             |
| 123 | Health             |

💡 Cette fonction peut aussi servir à supprimer les doublons dans un jeu de données

---

### 1.6.2.3 Sélectionner des colonnes avec la fonction `select()`

#### 1.6.2.3.1 Une seule colonne

```
# Sélection de la variable 'pub'
Journals |>
  dplyr::select(pub)
```

|     | pub                    |
|-----|------------------------|
| 1   | Blackwell              |
| 2   | So Afr ec history assn |
| 3   | Kluwer                 |
| 4   | Kluwer                 |
| 5   | Elsevier               |
| 6   | Elsevier               |
| 7   | Cambridge Univ Pres    |
| 8   | Elsevier               |
| 9   | Kluwer                 |
| ... |                        |

💡 On peut également utiliser la fonction `pull()` pour sélectionner une (et une seule) colonne

```
# Sélection de la variable 'pub'
```

```
Journals |>
```

```
dplyr::pull(pub)
```

```
[1] Blackwell      So Afr ec history assn
[3] Kluwer         Kluwer
[5] Elsevier       Elsevier
[7] Cambridge Univ Pres Elsevier
[9] Kluwer         Blackwell
[11] Kluwer         MIT
[13] Blackwell      Wiley
[15] Elsevier       Springer
[17] MIT Press      Intl Atlantic Ec. Soc.
[19] Sage           Elsevier
```

...

— Quelle différence avec `select()` ?

Le format de sortie est différent :

— la fonction `select()` renvoie la colonne sous forme de *dataframe*

— la fonction `pull()` renvoie une colonne sous forme de vecteur

### 1.6.2.3.2 Plusieurs colonnes

En listant la/les variables à conserver

```
# Sélection des variables "field" et "libprice"
```

```
Journals |>
```

```
dplyr::select(c(field, libprice))
```

```
      field libprice
1   General     123
2 Ec History     20
```

...

En listant la/les variables à supprimer

```
# Sélection des variables sauf "title" et "pub"
```

```
Journals |>
```

```
dplyr::select(-c(title, pub))
```

```
society libprice pages charpp citestot date1 oclc      field
1      no      123   440   3822      21 1986   14   General
2      no       20   309   1782      22 1986   59   Ec History
```

...

💡 Les tests sur le nom des variables peuvent se faire avec différentes fonctions :

- `contains()`
- `starts_with()`
- `ends_with()`
- `matches()`

```
# Sélection des variables dont le nom contient "s"
```

```
Journals |>  
  dplyr::select(contains("s"))
```

```
  society pages citestot  
1      no    440      21  
...
```

#### 1.6.2.4 Créer/ajouter de nouvelles variables avec `mutate()`

```
# Ajout de la variable 'anciennete'  
Journals |>  
  mutate(anciennete = 2026 - date1) -> Journals  
  
names(Journals)
```

```
[1] "title"      "pub"        "society"    "libprice"   "pages"  
[6] "charpp"     "citestot"   "date1"      "oclc"       "field"  
[11] "anciennete"
```

#### ⚠️ Penser à affecter la modification au jeu de données

- La fonction `mutate()` peut également
  - modifier une variable existante si le nom est le même que celui d'une colonne existante
  - supprimer une variable en fixant sa valeur à `NULL`

```
# Suppression de la variable 'charpp'  
Journals |>  
  mutate(charpp = NULL) |>  
  names()
```

```
[1] "title"      "pub"        "society"    "libprice"   "pages"  
[6] "citestot"   "date1"      "oclc"       "field"      "anciennete"
```

##### 1.6.2.4.1 Appliquer des règles conditionnelles

La fonction `if_else()`

```
Journals |>  
  mutate(cl_anciennete = if_else(anciennete < 30, " Moins de 30 ans",  
                                if_else(anciennete >= 30 & anciennete < 50, "30 à 49 ans",  
                                if_else(anciennete >= 50 & anciennete < 100, "50 à  
                                ↪ 99 ans",  
                                if_else(anciennete >= 100 & anciennete <  
                                ↪ 150,  
                                "100 à 149 ans", "150 ans et  
                                ↪ plus"))))) -> Journals
```

La fonction `case_when()`

```
Journals |>
  mutate(cl_anciennete = case_when(anciennete < 30 ~ "Moins de 30 ans",
    anciennete >= 30 & anciennete < 50 ~ "30 à 49 ans",
    anciennete >= 50 & anciennete < 100 ~ "50 à 99 ans",
    anciennete >= 100 & age < 150 ~ "100 à 149 ans",
    anciennete >= 150 ~ "150 ans et plus",
    TRUE ~ "ERREUR")) -> Journals
```

### 1.6.2.5 Réordonner les variables avec la fonction relocate()

💡 Les nouvelles variables ajoutées avec `mutate()` se positionnent toujours à la fin du jeu de données

» Il est possible de les positionner où l'on souhaite, avec la fonction `relocate()`

```
# On repositionne la variable 'anciennete' après la variable 'date1'
Journals |>
  relocate(anciennete, .after = date1) -> Journals

# Vérification
Journals |>
  names()
```

```
[1] "title"      "pub"        "society"    "libprice"   "pages"
[6] "charpp"    "citestot"   "date1"      "anciennete" "oclc"
[11] "field"
```

On aurait également pu obtenir le même résultat avec le paramètre `.before`

```
Journals |>
  relocate(anciennete, .before = oclc) -> Journals
```

### 1.6.2.6 Modification du nom des colonnes avec rename()

```
# On renomme la colonne 'pub' en 'publisher'
Journals |>
  dplyr::rename(publisher = pub) -> Journals

# Vérification : affichage du nom des colonnes
Journals |>
  names()
```

```
[1] "title"      "publisher"   "society"    "libprice"   "pages"
[6] "charpp"    "citestot"   "date1"      "anciennete" "oclc"
[11] "field"
```

⚠ Ne pas oublier d'affecter la modification au jeu de données

## 1.7 🎮 À VOUS DE JOUER!

**Exercice 1.2.** On se place toujours dans le projet `VisuR_exercices` et le fichier `01_Exercices_VisuR.R`, et on utilise les fonctions du `tidyverse`

1. Importer les données du fichier `communes.csv` dans un objet `communes`
2. Quel est le type de cet objet et sa structure (nombre de lignes et de colonnes)?
3. La variable `CODGEO` est un code. Transformer en variable caractère avec la fonction `as.character()`
4. Combien de communes distinctes y a-t-il dans `communes`? et combien d'années?

5. Filtrer les données pour n'afficher que les observations de l'année 2018
6. Créer la variable `surface = surface_urbaine + surface_verte` dans votre jeu de données. elle doit se trouver avant la variable `surface_urbaine`
7. Afficher la colonne commune et toutes celles qui commencent par "surface\_"

## Bibliographie

### Références citées

Orozco, V., Bontemps, C., Maigné, E., Piguet, V., Hofstetter, A., Lacroix, A., Levert, F., & Rousselle, J.-M. (2020). How To Make A Pie : Reproducible Research for Empirical Economics and Econometrics. *Journal of Economic Surveys*, 34(5), 11341169. <https://doi.org/10.1111/joes.12389>

### Packages

Bache, S. M., & Wickham, H. (2026). *magrittr* : A Forward-Pipe Operator for R. <https://magrittr.tidyverse.org>

Barbone, J. M., & Garbuszus, J. M. (2026). *openxlsx2* : Read, Write and Edit xlsx Files. <https://janmarvin.github.io/openxlsx2/>

Barnier, J., Briatte, F., & Larmarange, J. (2026). *questionr* : Functions to Make Surveys Processing Easier. <https://juba.github.io/questionr/>

Croissant, Y., & Graves, S. (2025). *Ecdat* : Data Sets for Econometrics. <https://www.r-project.org>

Csárdi, G., Hester, J., Wickham, H., Chang, W., Morgan, M., & Tenenbaum, D. (2024). *remotes* : R Package Installation from Remote Repositories, Including GitHub. <https://remotes.r-lib.org>

Dragulescu, A., & Arendt, C. (2020). *xlsx* : Read, Write, Format Excel 2007 and Excel 97/2000/XP/2003 Files. <https://github.com/colearendt/xlsx>

Grolemund, G., & Wickham, H. (2011). Dates and Times Made Easy with lubridate. *Journal of Statistical Software*, 40(3), 125. <https://www.jstatsoft.org/v40/i03/>

Hester, J., Angly, F., Chirico, M., Hyde, R., Kun, R., Patil, I., & Rosenstock, A. (2025). Static Code Analysis for R. *Journal of Open Source Software*, 10(108), 7240. <https://doi.org/10.21105/joss.07240>

Hester, J., Angly, F., Hyde, R., Chirico, M., Ren, K., Rosenstock, A., Patil, I., Gruson, H., & Ferdman, E. (2026). *lintr* : A Linter for R Code. <https://lintr.r-lib.org>

Müller, K., Walthert, L., & Patil, I. (2025). *styler* : Non-Invasive Pretty Printing of R Code. <https://github.com/r-lib/styler>

Müller, K., & Wickham, H. (2026). *tibble* : Simple Data Frames. <https://tibble.tidyverse.org/>

Ooms, J. (2025). *writexl* : Export Data Frames to Excel xlsx Format. <https://ropensci.r-universe.dev/writexl>

R Core Team. (2026). *foreign* : Read Data Stored by Minitab, S, SAS, SPSS, Stata, Systat, Weka, dBase, ... <https://svn.r-project.org/R-packages/trunk/foreign/>

Spinu, V., Grolemund, G., & Wickham, H. (2026). *lubridate* : Make Dealing with Dates a Little Easier. <https://lubridate.tidyverse.org>

Walthert, L. (2026). *strcode* : Structure and abstract your code. <https://github.com/lorenzwalther/strcode>

Wickham, H. (2016). *ggplot2* : Elegant Graphics for Data Analysis. Springer-Verlag New York. <https://ggplot2.tidyverse.org>

Wickham, H. (2023). *tidyverse* : Easily Install and Load the Tidyverse. <https://tidyverse.tidyverse.org>

Wickham, H. (2025a). *forcats* : Tools for Working with Categorical Variables (Factors). <https://forcats.tidyverse.org/>

Wickham, H. (2025b). *stringr* : Simple, Consistent Wrappers for Common String Operations. <https://stringr.tidyverse.org>

Wickham, H., Averick, M., Bryan, J., Chang, W., McGowan, L. D., François, R., Grolemund, G., Hayes, A., Henry, L., Hester, J., Kuhn, M., Pedersen, T. L., Miller, E., Bache, S. M., Müller, K., Ooms, J., Robinson, D., Seidel, D. P., Spinu, V., ... Yutani, H. (2019). Welcome to the tidyverse. *Journal of Open Source Software*, 4(43), 1686. <https://doi.org/10.21105/joss.01686>

Wickham, H., & Bryan, J. (2026). *readxl* : Read Excel Files. <https://readxl.tidyverse.org>

Wickham, H., Chang, W., Henry, L., Pedersen, T. L., Takahashi, K., Wilke, C., Woo, K., Yutani, H., Dunnington, D., & van den Brand, T. (2026). *ggplot2* : Create Elegant Data Visualisations Using the Grammar of Graphics. <https://ggplot2.tidyverse.org>

Wickham, H., François, R., Henry, L., Müller, K., & Vaughan, D. (2026). *dplyr* : A Grammar of Data Manipulation. <https://dplyr.tidyverse.org>

Wickham, H., & Henry, L. (2026). *purrr* : Functional Programming Tools. <https://purrr.tidyverse.org/>

Wickham, H., Hester, J., & Bryan, J. (2026). *readr* : Read Rectangular Text Data. <https://readr.tidyverse.org>

Wickham, H., Miller, E., & Smith, D. (2025). *haven : Import and Export SPSS, Stata and SAS Files*. <https://haven.tidyverse.org>

Wickham, H., Vaughan, D., & Girlich, M. (2025). *tidyr : Tidy Messy Data*. <https://tidyr.tidyverse.org>